Relatório de Implementação: Algoritmo de Kruskal

Oct 19, 2026

Curso: Ciência da Computação
Atividade Acadêmica: Análise e Projeto de Algoritmos (2026/2)
Professora: Maria Adelina Raupp Sganzerla

Felipe Flores - felipeflores0781@gmail.com
Leandro Berwanger - berwangerleandro@edu.unisinos.br

Descrição da Atividade

A equipe implementou em C++17 o algoritmo de Kruskal, que encontra a Árvore Geradora Mínima (AGM) de um grafo não direcionado, ponderado e conexo. A estratégia é gulosa: as arestas são examinadas da mais leve para a mais pesada e cada uma é aceita se não formar ciclo com as já escolhidas. A ordenação usa um Merge Sort próprio e a detecção de ciclos usa a estrutura Union-Find com compressão de caminho e união por rank.

A implementação foi testada em três instâncias (7 vértices e 11 arestas; 1.000 vértices e 10.000 arestas; 1.000.000 de vértices e 10.000.000 de arestas) e conferida contra uma implementação independente do algoritmo de Prim em 1.000 grafos aleatórios, sem nenhuma divergência. A instância grande é resolvida em cerca de 0,6 s, dos quais 91% são gastos na ordenação. O relatório também mede o efeito das otimizações do Union-Find, da parada antecipada e da densidade do grafo.

Fundamentação Teórica

Descrição do problema

Dado um grafo G = (V, E) não direcionado, conexo e com um peso w(e) em cada aresta, uma árvore geradora é um subconjunto de |V| − 1 arestas que conecta todos os vértices sem formar ciclos. A Árvore Geradora Mínima é a árvore geradora de menor soma de pesos (CORMEN et al., 2012, cap. 23). Kruskal publicou o algoritmo em 1956 como solução para esse problema (KRUSKAL, 1956).

Aplicações práticas

- **Projeto de redes:** ligar cidades, prédios ou equipamentos com o menor custo total de cabo, fibra, tubulação ou estrada.

- **Agrupamento (clustering):** executar o Kruskal e parar com k componentes dá o agrupamento de ligação simples, que maximiza o espaçamento entre grupos (KLEINBERG; TARDOS, 2006, seção 4.7).
- **Segmentação de imagens:** pixels são vértices e a diferença de cor é o peso; regiões são unidas da aresta mais leve para a mais pesada.
- **Aproximação para o Caixeiro Viajante métrico:** percorrer a AGM em pré-ordem dá uma rota com no máximo o dobro do custo ótimo (CORMEN et al., 2012, seção 35.2).
- **Geração de labirintos:** pesos aleatórios nas paredes produzem um labirinto com caminho único entre quaisquer duas células.

Entradas e saídas

- **Entrada:** o número de vértices n e a lista de m arestas (u, v, peso). O grafo precisa ser conexo.
- **Saída:** as n − 1 arestas da AGM e o seu peso total. Se o grafo for desconexo, o programa informa que não existe árvore geradora.

Estratégia gulosa e critério de escolha local

A cada passo o algoritmo escolhe a aresta de menor peso ainda não examinada. Ela é aceita se ligar duas componentes diferentes da floresta construída até ali; caso contrário fecharia um ciclo e é descartada. A floresta começa com n árvores de um vértice e termina como uma única árvore.

Por que a escolha gulosa é adequada

A justificativa é a **propriedade do corte**: para qualquer divisão dos vértices em dois grupos, a aresta mais leve que cruza a divisão pertence a alguma AGM (CORMEN et al., 2012, Teorema 23.1). Quando o Kruskal aceita a aresta (u, v), ela é a mais leve que sai da componente de u, porque todas as mais leves já foram examinadas. Logo, cada aceitação é segura e nunca precisa ser desfeita.

Quando a solução é ótima

O Kruskal produz uma AGM ótima em **todo** grafo conexo, com pesos quaisquer, inclusive negativos e repetidos. Com pesos repetidos podem existir várias AGMs diferentes, todas com o mesmo peso total; o algoritmo devolve uma delas. Com todos os pesos distintos, a AGM é única.

Complexidade

- **Ordenação:** $O(m \log m)$, que equivale a $O(m \log n)$, pois $m \leqslant n^2$.

- **Seleção:** no máximo m iterações, cada uma com duas operações find de custo amortizado O(α(n)), onde α é a inversa da função de Ackermann e vale no máximo 4 na prática (TARJAN, 1975).
- **Total:** O(m log m), dominado pela ordenação.
- **Espaço:** O(n + m): a lista de arestas, o vetor auxiliar do Merge Sort e os vetores pai e rank do Union-Find.

Estruturas de dados

- **Lista de arestas** (vetor contíguo): o Kruskal só percorre arestas em ordem de peso e nunca consulta vizinhos, então não precisa de lista ou matriz de adjacência.
- **Union-Find (conjuntos disjuntos):** representa as componentes da floresta e responde se dois vértices já estão conectados (CORMEN et al., 2012, cap. 21).

Projeto de Algoritmo

Estrutura de dados utilizada: um vetor contíguo de `Aresta {peso, u, v}` (12 bytes por aresta) e um Union-Find com os vetores `pai` e `rank`. O vetor de arestas é a forma mais compacta para um algoritmo que só percorre arestas em ordem, e é amigável ao cache do processador. O Union-Find responde "u e v já estão conectados?" em tempo amortizado praticamente constante; sem ele, cada teste de ciclo exigiria uma busca no grafo, com custo O(n).

Funções/métodos utilizados:

Função	Arquivo	Responsabilidade
`kruskal(n, arestas, opcoes)`	kruskal.hpp	Ordena, executa o laço guloso e devolve a árvore, o peso e as métricas
`mergeSort(arestas)`	ordenacao.hpp	Ordena as arestas por peso e conta as comparações
`UnionFind::find(x)`	estruturas.hpp	Devolve a raiz da componente de x, comprimindo o caminho
`UnionFind::unir(a, b)`	estruturas.hpp	Une duas componentes por rank; devolve falso se já estavam unidas
`gerarGrafoConexo(n, m, semente)`	grafos.hpp	Gera grafos aleatórios conexos e reproduzíveis
`lerArquivo(caminho)`	grafos.hpp	Lê um grafo em texto, com vértices numéricos ou nomeados

Função	Arquivo	Responsabilidade
`primPesoTotal(n, arestas)`	prim.hpp	Algoritmo independente usado apenas para conferir o resultado

A separação em arquivos isola cada responsabilidade, e o Kruskal recebe as opções (Union-Find otimizado ou ingênuo, parada antecipada) por parâmetro, o que permite medir cada otimização sem duplicar código.

Estratégia de randomização: o gerador primeiro embaralha os vértices e liga cada um a um vértice anterior sorteado, formando uma árvore aleatória que garante a conexidade. Depois acrescenta arestas sorteadas, sem laços e sem repetições, até atingir m; em grafos com mais da metade das arestas possíveis, sorteia entre os pares que faltam para evitar rejeições. Os pesos são inteiros uniformes de 1 a 1.000 e a lista final é embaralhada, para que a ordem de entrada não favoreça a ordenação. O sorteio usa `std::mt19937_64` com semente fixa e embaralhamento de Fisher-Yates próprio, pois as distribuições da biblioteca padrão variam entre compiladores; assim a mesma semente gera o mesmo grafo em qualquer máquina.

Algoritmo de ordenação: Merge Sort bottom-up (iterativo), implementado pela equipe. Ele garante O(m log m) no pior caso, sem o pior caso O(m^2) do Quick Sort, e é estável: arestas de mesmo peso mantêm a ordem de entrada, o que deixa a execução determinística e fácil de acompanhar no teste passo a passo. A versão iterativa evita recursão profunda. O custo é um vetor auxiliar de m arestas.

Análise de otimização: foram aplicadas e medidas três otimizações (resultados na seção de Verificação):

1. **Compressão de caminho e união por rank** no Union-Find: reduzem o custo do find de O(n) no pior caso para O(α(n)) amortizado.
2. **Parada antecipada:** o laço termina quando a árvore atinge n − 1 arestas, sem examinar as arestas mais pesadas que restam.
3. **Medição da ordenação separada da seleção:** mostrou que a ordenação domina o tempo e que é nela que um ganho futuro faria diferença.

IA utilizada no auxílio da implementação: Claude (modelo Claude Opus 5.5, da Anthropic), usado no app Claude. Prompts usados pela equipe: "preciso realizar esse trabalho e escolher qual algoritmo guloso irei escolher" (com o enunciado e o template anexados), "Vamos fazer o algoritmo de kruskal" e "Faça o código em C++". A IA ajudou na escolha do tema, na estrutura do código, nos casos de teste e na redação deste relatório. Justificativa: acelerar a implementação e a organização dos testes; a equipe revisou o código, conferiu os resultados contra o algoritmo de Prim e é responsável pelo conteúdo entregue.

Descrição do Algoritmo

1. **Dados de entrada:** n vértices (numerados de 0 a n − 1, ou nomeados quando lidos de arquivo) e uma lista de m arestas (peso, u, v). Os dados vêm de um arquivo texto ou do gerador aleatório.
2. **Organização inicial:** a lista de arestas é ordenada por peso crescente com o Merge Sort. O Union-Find é criado com n conjuntos, um por vértice: no início, cada vértice é uma componente isolada.
3. **Critério de escolha gulosa:** a próxima aresta da lista ordenada, ou seja, a mais leve ainda não examinada.
4. **Seleção ou descarte:** para a aresta (u, v), o algoritmo chama `unir(u, v)`, que calcula `find(u)` e `find(v)`.
   - Raízes diferentes: u e v estão em componentes distintas; as componentes são unidas e a aresta **entra** na árvore.
   - Mesma raiz: u e v já estão conectados; a aresta fecharia um ciclo e é **descartada**.
5. **Condição de parada:** a árvore atinge n − 1 arestas (parada antecipada) ou a lista termina. Se a lista terminar com menos de n − 1 arestas aceitas, o grafo é desconexo e o programa informa o erro.
6. **Solução final:** as arestas aceitas, na ordem em que foram escolhidas, e a soma dos seus pesos. O programa também imprime as métricas: arestas examinadas e descartadas, comparações da ordenação, chamadas de find e tempos.

Pseudocódigo da implementação:

```
KRUSKAL(n, arestas):
    MERGE-SORT(arestas) por peso crescente
    conjuntos <- UNION-FIND(n)
    arvore <- lista vazia; peso_total <- 0
    para cada (peso, u, v) em arestas:
        se conjuntos.UNIR(u, v):          // u e v estavam separados
            arvore.adiciona((peso, u, v))
            peso_total <- peso_total + peso
            se |arvore| = n - 1: pare     // parada antecipada
        // senao: descartada, formaria ciclo
    se |arvore| != n - 1: erro "grafo desconexo"
    retorne arvore, peso_total
```

Codificação/Implementação/Ambiente

Linguagem/versão utilizada: C++17. A linguagem dá controle direto sobre a memória (vetores contíguos de arestas) e gera código nativo, o que permite medir instâncias de 10 milhões de arestas em cerca de 0,6 s. O padrão C++17 traz recursos usados no código, como desestruturação (`auto [peso, v]`), e é suportado por todos os compiladores atuais. Só a biblioteca padrão é usada, e nenhuma função dela resolve a AGM.

Compilador/interpretador: GNU g++ 16.2.0 (distribuição WinLibs, MinGW-w64) no Windows, com as opções `-std=c++17 -O2`. O `-O2` é o nível usual para medir desempenho, e o WinLibs é a forma mais simples de ter o g++ atualizado no Windows. Durante o desenvolvimento, o código também foi compilado com g++ 13.3 e clang 18 no Linux, com `-Wall -Wextra -pedantic`, sem nenhum aviso; o `Makefile` do repositório usa essas opções.

Hardware utilizado para implementação: desktop com processador AMD Ryzen 7 5700X (8 núcleos, 16 threads, 3,4 GHz, 32 MB de cache L3) e 16 GB de RAM. É o computador da equipe; o programa é sequencial e usa um único núcleo, então o desempenho depende da velocidade de um núcleo e do cache, não da quantidade de núcleos. Todos os testes deste relatório rodaram nesta máquina. Durante o desenvolvimento, a bateria também rodou numa máquina virtual Linux (Intel Xeon 2,1 GHz, 2 núcleos, 7,8 GB), usada nas comparações entre máquinas. O teste grande usa cerca de 600 MB de RAM no pico.

Sistema Operacional: Windows, 64 bits (x64). É o sistema do computador da equipe; o código usa só a biblioteca padrão do C++ e também roda em Linux e macOS.

Link para o código: [github.com/LeandroDQuevedo/Unisinos, pasta Aula 9 - Algoritmos Gulosos]

Para reproduzir no Windows: `g++ -std=c++17 -O2 -o kruskal.exe src\main.cpp` e depois `kruskal.exe todos`. No Linux ou macOS: `make` e depois `./kruskal todos`. A execução completa leva cerca de 1 minuto. O arquivo `resultados/saida_completa.txt` do repositório contém a saída usada neste relatório.

Verificação e Análise dos Dados

O algoritmo produziu a árvore correta em todos os testes: nos três casos de teste e em 1.000 grafos aleatórios, o peso obtido foi igual ao do algoritmo de Prim, sem nenhuma divergência. Os testes rodaram no computador da equipe, descrito acima. Durante o desenvolvimento a bateria também rodou numa máquina virtual Linux: as mesmas sementes geraram os mesmos grafos, com as mesmas decisões e os mesmos pesos nas duas máquinas, o que confirma que os testes são reproduzíveis. Os tempos são a mediana

de várias execuções sobre a mesma entrada; a geração do grafo não entra na medição. Métricas usadas: tempo (ordenação, seleção e total, com mínimo, máximo e desvio padrão), comparações da ordenação, chamadas de find e arestas examinadas.

Teste 1: pequena instância (passo a passo)

Entrada: arquivo `entradas/pequeno.txt`, com 7 vértices (A a G) e 11 arestas: A-B 7, A-D 5, B-C 8, B-D 9, B-E 7, C-E 5, D-E 15, D-F 6, E-F 8, E-G 9, F-G 11.

Decisões do algoritmo (arestas já ordenadas por peso):

Passo	Aresta	Peso	Decisão	Motivo	Peso acumulado
1	A-D	5	Aceita	A e D em componentes distintas	5
2	C-E	5	Aceita	C e E em componentes distintas	10
3	D-F	6	Aceita	F se junta a {A, D}	16
4	A-B	7	Aceita	B se junta a {A, D, F}	23
5	B-E	7	Aceita	Une {A, B, D, F} e {C, E}	30
6	B-C	8	Descarta	B e C já conectados (B-E-C)	30
7	E-F	8	Descarta	E e F já conectados (E-B-A-D-F)	30
8	B-D	9	Descarta	B e D já conectados (B-A-D)	30
9	E-G	9	Aceita	G se junta à árvore; 6 = n − 1 arestas	39

Solução obtida: A-D (5), C-E (5), D-F (6), A-B (7), B-E (7), E-G (9), peso total **39**, confirmado pelo Prim.

Tempo de execução: abaixo de 0,01 ms, pequeno demais para uma medição significativa.

Análise: o algoritmo parou no passo 9 com a árvore completa e não examinou D-E (15) nem F-G (11). Os empates (5 e 5, 7 e 7, 8 e 8, 9 e 9) foram resolvidos pela ordem do arquivo, graças à estabilidade do Merge Sort. Os três descartes mostram o Union-Find detectando ciclos que não são visíveis olhando só a aresta.

Teste 2: média instância

Entrada: grafo aleatório com 1.000 vértices e 10.000 arestas (semente 7, pesos de 1 a 1.000).

Decisões realizadas: 3.429 decisões, 999 aceitas e 2.430 descartadas. Todas as arestas de peso 1 a 50 foram aceitas; o primeiro descarte foi a aresta 874-374, de peso 51, quando a floresta já tinha componentes grandes o bastante para formar ciclos. A listagem completa das 20 primeiras decisões está em `resultados/saida_completa.txt`.

Solução obtida: 999 arestas, peso total **62.325**, igual ao do Prim. A aresta mais pesada da árvore tem peso 340, embora o grafo tenha arestas de até 1.000.

Métricas: 123.575 comparações na ordenação, 6.858 chamadas de find e 3.429 arestas examinadas (34,3% do total).

Tempo de execução: mediana de 0,428 ms em 10 execuções (mínimo 0,426 ms, máximo 0,454 ms, desvio padrão 0,009 ms). Na primeira execução, a ordenação levou 0,40 ms e a seleção 0,09 ms.

Análise: a parada antecipada evitou examinar 66% das arestas, porque a árvore se completou com peso máximo 340. A ordenação ocupou cerca de 80% do tempo.

Teste 3: grande instância

Entrada: grafo aleatório com 1.000.000 de vértices e 10.000.000 de arestas (semente 2026). A geração levou 2,7 s, fora da medição.

Decisões realizadas: 6.517.008 arestas examinadas (65,2%), 999.999 aceitas e 5.517.009 descartadas por ciclo.

Solução obtida: 999.999 arestas, peso total **60.560.622**, igual ao do Prim.

Métricas: 223.950.078 comparações na ordenação (próximo de $m \cdot \log_2 m \approx 232$ milhões) e 13.034.016 chamadas de find (exatamente duas por aresta examinada).

Tempo de execução (mediana de 5 execuções):

Etapa	Mediana (ms)	Mínimo (ms)	Máximo (ms)	Desvio (ms)
Ordenação	551,1	547,9	567,7	8,0
Seleção gulosa	53,3	53,1	54,5	0,6
Total	604,7	601,1	622,2	8,5

Análise: 10 milhões de arestas foram processadas em 0,6 s, com 91% do tempo na ordenação. A seleção, mesmo examinando 6,5 milhões de arestas, custou só 53 ms, o que confirma o custo quase constante do Union-Find otimizado. O Prim levou 3,3 s na mesma entrada, 5,4 vezes mais.

Verificação da correção

Além dos três testes, o modo `verificacao` gerou 1.000 grafos aleatórios com 1 a 80 vértices e densidades variadas; metade usa pesos de 1 a 10, o que força muitos empates. Em cada grafo foram conferidos: o mesmo peso total do Prim, exatamente n − 1 arestas e todos os vértices alcançáveis pela árvore (o que, com n − 1 arestas, garante ausência de

ciclos). Resultado: **0 falhas em 1.000 grafos**. Também foram testados casos-limite: grafo desconexo (o programa recusa com mensagem de erro), grafo com um vértice (árvore vazia, peso 0) e todas as arestas com o mesmo peso.

Crescimento do tempo com o tamanho

Grafos com m = 10n, mediana de 5 execuções:

n	m	Ordenação (ms)	Seleção (ms)	Total (ms)	Tempo por aresta (ns)	Arestas examinadas
1.000	10.000	0,44	0,02	0,47	47	36,3%
10.000	100.000	4,37	0,30	4,66	47	52,6%
100.000	1.000.000	45,41	4,38	49,78	50	79,8%
1.000.000	10.000.000	558,96	54,25	613,02	61	64,7%

Cada vez que m é multiplicado por 10, o tempo total cresce 10,0, 10,7 e 12,3 vezes. O tempo por aresta fica estável em 47 a 50 ns até 1 milhão de arestas e sobe para 61 ns com 10 milhões. **Hipóteses:** até 1 milhão de arestas (12 MB), o custo é dominado por passadas sequenciais na memória, que o processador antecipa bem, e o fator log m (de 13 para 20) quase não aparece. Com 10 milhões de arestas, os 120 MB do vetor mais os 120 MB do auxiliar do Merge Sort não cabem nos 32 MB de cache L3 do Ryzen 7 5700X e cada passada busca os dados na RAM; com 1 milhão de arestas, vetor e auxiliar somam 24 MB e ainda cabem. Na máquina virtual usada no desenvolvimento, a linha de 100 mil arestas tinha ficado fora da curva (103 ns por aresta); no computador da equipe ela ficou alinhada com as outras, o que confirma que aquela medição era ruído da máquina virtual. A fração de arestas examinadas varia de 36% a 80% porque depende do peso da aresta mais pesada da árvore, que muda de grafo para grafo.

Efeito das otimizações

Union-Find otimizado x ingênuo. A versão ingênua não usa compressão de caminho nem união por rank. Tempo só da seleção (mediana de 3 execuções):

Grafo	n	Otimizado (ms)	Ingênuo (ms)	Ingênuo / otimizado
Estrela (caso adverso)	5.000	0,025	16,4	670×
Estrela (caso adverso)	10.000	0,063	65,9	1.044×
Estrela (caso adverso)	20.000	0,138	259,4	1.881×
Estrela (caso adverso)	40.000	0,260	1.037,1	3.986×

Grafo	n	Otimizado (ms)	Ingênuo (ms)	Ingênuo / otimizado
Aleatório (m = 10n)	5.000	0,148	32,7	222×
Aleatório (m = 10n)	10.000	0,302	151,7	502×
Aleatório (m = 10n)	20.000	0,592	961,3	1.625×
Aleatório (m = 10n)	40.000	1,472	5.043,1	3.426×

No grafo estrela, todas as arestas tocam o vértice 0, e a versão ingênua transforma a árvore do Union-Find numa lista: cada vez que n dobra, o tempo quadruplica (16, 66, 259, 1.037 ms), exatamente o comportamento $O(n^2)$ esperado. No grafo aleatório, a versão ingênua foi ainda mais lenta que no caso adverso, o que motivou a hipótese da seção seguinte. A versão otimizada ficou abaixo de 1,5 ms em todos os casos.

Parada antecipada. Em um grafo com 200.000 vértices e 2.000.000 de arestas, a parada examinou 1.208.643 arestas em vez de 2.000.000 (40% a menos), e a seleção caiu de 10,9 ms para 8,7 ms (20% a menos). **Hipótese:** o ganho é metade do que a redução de arestas sugere porque, depois que a árvore fica completa, toda a floresta é um único conjunto já comprimido, e cada find restante custa um ou dois acessos à memória; as arestas evitadas eram as mais baratas. O ganho real seria evitar a ordenação dessas arestas, o que exigiria trocar a ordenação completa por um heap.

Ordenação. Com 2.000.000 de arestas, o Merge Sort próprio levou 99,1 ms e o `std::sort` da biblioteca, usado só como referência, 60,0 ms. Os dois são $O(m \log m)$; a diferença vem de o introsort ordenar no próprio vetor, sem cópia para um auxiliar, aproveitando melhor o cache.

Hipótese levantada durante o desenvolvimento

O que aconteceu. A primeira execução completa dos testes não terminou em 10 minutos. A investigação mostrou que o programa estava parado na medição do Union-Find ingênuo, num grafo aleatório de 200.000 vértices e 2.000.000 de arestas. Isso surpreendeu a equipe: o grafo estrela tinha sido montado justamente para ser o pior caso, e o grafo aleatório deveria ser um caso "típico", mais leve. Mesmo com 20.000 vértices, o aleatório levou 1.489 ms contra 511 ms da estrela; no computador da equipe a diferença se repetiu (961 ms contra 259 ms).

Hipótese. Sem união por rank, a versão ingênua pendura uma raiz na outra sem olhar o tamanho das árvores. Numa ordem aleatória de arestas, muitas vezes a componente gigante acaba pendurada embaixo de um vértice isolado, e todos os seus vértices ganham um nível de profundidade. A floresta fica funda e cada find precisa subir esses níveis. Se a

hipótese estiver certa: (1) trocar o lado em que se pendura não deve resolver; (2) só a união por rank, mesmo sem compressão, deve resolver; (3) uma fração grande das uniões deve pendurar a árvore maior sob a menor.

Experimento. O modo `./kruskal hipotese` roda o laço do Kruskal com cinco variantes do Union-Find, instrumentadas para contar os ponteiros percorridos em cada find e a altura final da floresta. Mesmo grafo aleatório da seção anterior (n = 20.000, m = 200.000), mediana de 3 execuções:

Variante	Tempo (ms)	Passos por find	Altura final	Uniões que penduraram a árvore maior sob a menor
Ingênuo (v sob u)	1.095,1	1.999,0	3.707	36,9%
Ingênuo invertido (u sob v)	1.032,6	1.876,2	3.526	36,1%
Só união por rank	1,71	1,86	6	2,1%
Só compressão de caminho	1,33	2,23	3	36,9%
Otimizado (rank + compressão)	0,58	0,95	2	2,1%

Medido no computador da equipe; os tempos incluem os contadores do experimento. As contagens de passos, finds e altura foram idênticas às obtidas na máquina virtual do desenvolvimento, porque dependem só do grafo.

Resultado. As três previsões se confirmaram. Trocar o lado não ajudou: as duas versões ingênuas ficaram com cerca de 2.000 passos por find e árvores de mais de 3.500 níveis, porque em ambas mais de um terço das uniões pendurou a árvore maior sob a menor. Só a união por rank derrubou a altura para 6 e o custo para menos de 2 passos por find, 639 vezes mais rápido. A compressão sozinha também resolveu, por outro caminho: ela não evita a união ruim, mas achata a árvore na primeira consulta.

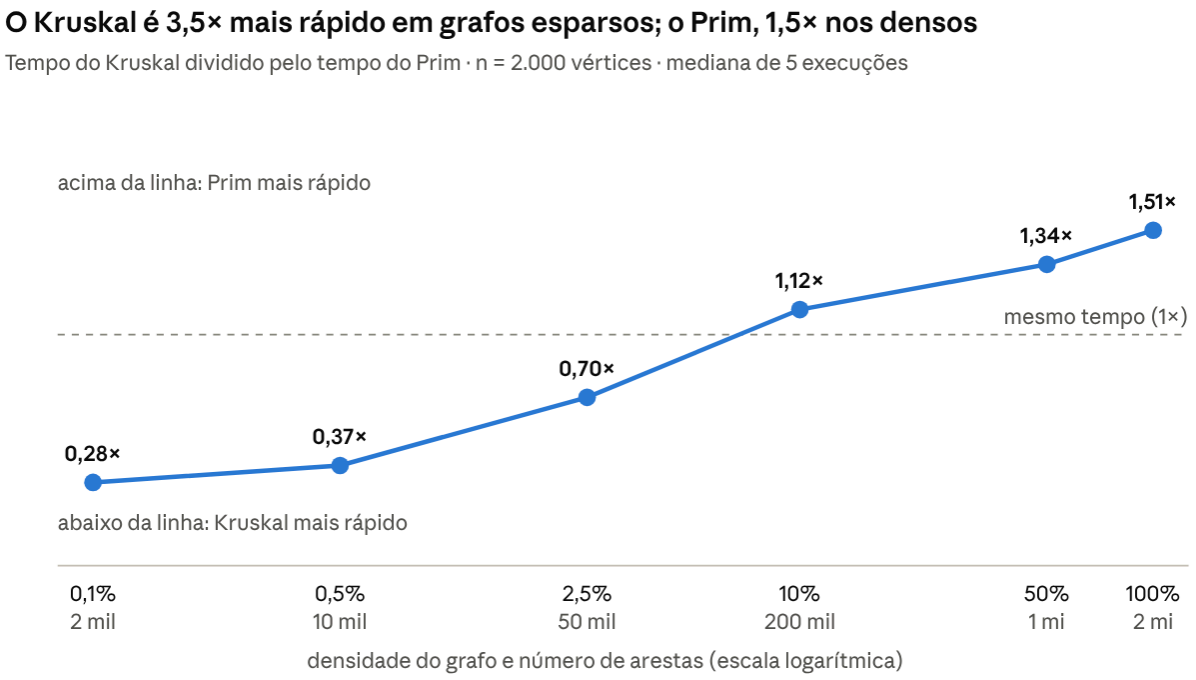
Por que o aleatório ficou pior que a estrela. Na estrela a árvore fica ainda mais funda (19.999 níveis, 5.000 passos por find), mas o algoritmo faz só 39.998 finds. No grafo aleatório ele examina muito mais arestas até completar a árvore, e cada descarte também custa dois finds: foram 206.474 finds, cinco vezes mais. São cerca de 410 milhões de passos contra 200 milhões na estrela.

Uma descoberta extra. Na estrela, a versão ingênua invertida levou 0,12 ms em vez de 346 ms. O "pior caso" que montamos dependia de a união sempre pendurar para o mesmo lado. Já o grafo aleatório é ruim para qualquer escolha de lado que ignore o tamanho.

Conclusão: a ordem aleatória das arestas não protege o Union-Find ingênuo; a proteção vem das heurísticas, e qualquer uma das duas basta para evitar o desastre.

Grafos esparsos x densos

Com o número de vértices fixo em 2.000 e o número de arestas crescendo do mínimo ao grafo completo, o Kruskal é até 3,5 vezes mais rápido nos grafos esparsos e o Prim é até 1,5 vez mais rápido nos densos; as curvas se cruzam entre 2,5% e 10% de densidade. No grafo completo, o Kruskal levou 98,0 ms e o Prim 65,1 ms.



resultados/saida_completa.txt · modo densidade, 6 grafos com 2.000 vértices

Hipótese: o Kruskal ordena todas as m arestas antes de decidir qualquer coisa, mas no grafo completo examinou só 7.166 delas (0,4%) até completar a árvore; quase todo o trabalho de ordenação foi desperdiçado. O Prim só coloca no heap as arestas que saem da árvore em construção, e em grafo denso a maioria aponta para vértices já incluídos e é ignorada sem custo de ordenação. Nos grafos esparsos, a ordenação é barata e o laço do Kruskal com vetor contíguo é mais leve que as operações de heap do Prim.

Comparação entre as máquinas. Na máquina virtual do desenvolvimento, o Prim chegou a ser 2 vezes mais rápido no grafo completo; no computador da equipe, só 1,5 vez. Do Linux para o computador da equipe, o Kruskal ficou 1,85 vez mais rápido nesse grafo (180,9 para 98,0 ms) e o Prim só 1,4 vez (91,5 para 65,1 ms). **Hipótese:** o Merge Sort percorre a memória em sequência e aproveita a maior largura de banda e a antecipação de dados do processador mais novo, enquanto o heap e as listas de adjacência do Prim fazem acessos espalhados, limitados pela latência da memória, que melhora menos entre máquinas.

Análise Crítica

Qual é exatamente a escolha gulosa realizada pelo algoritmo? Escolher, entre as arestas ainda não examinadas, a de menor peso, e aceitá-la se ela ligar duas componentes diferentes da floresta atual.

Por que essa escolha parece ser a melhor decisão naquele momento? É a forma mais barata de reduzir em um o número de componentes. Pela propriedade do corte, a aresta mais leve que sai de uma componente pertence a alguma AGM; como todas as arestas mais leves já foram examinadas, a aresta aceita é exatamente essa aresta mínima de um corte.

O algoritmo reconsidera decisões anteriores? Por quê? Não. Uma aresta aceita nunca sai da árvore e uma descartada nunca volta. A aceita é segura pela propriedade do corte. A descartada fecharia um ciclo com arestas de peso menor ou igual, e pela propriedade do ciclo a aresta mais pesada de um ciclo pode ser excluída sem perder a otimalidade.

Existe subestrutura ótima no problema? Explique. Sim. Se T é uma AGM de G e a aresta (u, v) pertence a T, contrair u e v num único vértice transforma T − (u, v) numa AGM do grafo contraído. Cada aceitação do Kruskal equivale a essa contração: o que resta resolver é o mesmo problema, menor, sobre as componentes.

O problema apresenta a propriedade da escolha gulosa? Sim. Existe sempre uma AGM que contém a aresta de menor peso do grafo (e, mais geralmente, a aresta mínima de qualquer corte). A prova é por troca: se uma AGM T não contém a aresta mínima e, acrescentá-la cria um ciclo; trocando e por outra aresta do ciclo que cruza o mesmo corte obtém-se uma árvore de peso menor ou igual.

A solução obtida é sempre ótima? Sim, para qualquer grafo conexo com pesos reais, inclusive negativos e repetidos. Os testes confirmaram isso em 1.003 grafos, sem divergência em relação ao Prim. Com pesos repetidos podem existir várias AGMs; o algoritmo devolve uma delas, sempre com o peso mínimo.

Em quais condições o algoritmo pode não produzir a melhor solução? O Kruskal não falha no problema para o qual foi feito, mas não serve para variações dele:

- **Grafo desconexo:** não existe árvore geradora; o algoritmo produz uma floresta geradora mínima, e a implementação informa o erro.
- **Grafo direcionado:** a AGM direcionada (arborescência mínima) exige o algoritmo de Chu-Liu/Edmonds; a escolha gulosa por aresta pode gerar um resultado inválido.
- **Restrições extras:** limite de grau por vértice, limite de diâmetro ou árvore de Steiner (ligar só alguns vértices) são problemas NP-difíceis, e a escolha gulosa deixa de ser ótima.

- **Outro objetivo:** se o que se quer é o menor caminho entre dois pontos, a AGM não serve; o problema certo é o de Dijkstra.

Qual é a complexidade computacional da implementação? O(m log m) de tempo: O(m log m) do Merge Sort mais O(m · α(n)) do laço com Union-Find. Espaço O(n + m). Os testes confirmam que a ordenação domina: 91% do tempo na instância grande, com 223,9 milhões de comparações, próximo das $m \cdot \log_2 m \approx 232$ milhões previstas.

Qual seria uma abordagem alternativa para resolver o mesmo problema?

- **Prim:** cresce uma única árvore a partir de um vértice, sempre pela aresta mais leve que sai dela, usando heap. O(m log n) com heap binário, ou O(m + n log n) com heap de Fibonacci. Nos testes foi até 1,5 vez mais rápido que o Kruskal em grafos densos e até 3,5 vezes mais lento em esparsos.
- **Borůvka:** a cada rodada, cada componente escolhe sua aresta mais leve de saída; O(m log n), fácil de paralelizar.
- **Kruskal com heap:** troca a ordenação completa por um heap e extrai só as arestas necessárias, aproveitando a parada antecipada em grafos densos.

Quais são as principais vantagens e limitações da estratégia gulosa utilizada?

- **Vantagens:** ótima garantida com prova simples; implementação curta; só precisa da lista de arestas, sem montar adjacências; rápida em grafos esparsos; a ordem de aceitação já fornece o agrupamento por ligação simples.
- **Limitações:** precisa ordenar todas as arestas antes de decidir, o que desperdiça trabalho em grafos densos (examinou só 0,4% das arestas do grafo completo); depende do Union-Find otimizado, sem o qual o laço chegou a ser quase 4.000 vezes mais lento; não se adapta a restrições extras nem a grafos direcionados.

Considerações Finais

O Kruskal é um caso em que a escolha gulosa é comprovadamente ótima: a propriedade do corte garante que cada aresta aceita pertence a uma AGM, e os testes confirmaram isso em mais de mil grafos, sem divergência em relação ao Prim. A implementação em C++ resolveu um grafo de 10 milhões de arestas em 0,6 s, e as medições mostraram onde está o custo real: 91% do tempo na ordenação, enquanto o laço guloso, que é a parte "inteligente" do algoritmo, é quase gratuito graças ao Union-Find.

A atividade também mostrou que a mesma ideia gulosa pode ter desempenho muito diferente conforme as estruturas de dados escolhidas. Sem compressão de caminho e união por rank, o laço ficou milhares de vezes mais lento; em grafos densos, o Prim foi até

1,5 vez mais rápido porque não precisa ordenar arestas que nunca usa. Escolher um algoritmo guloso, portanto, envolve duas perguntas: se a escolha local leva ao ótimo e se os dados permitem fazer essa escolha de forma eficiente.

Referências Bibliográficas

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012. Caps. 21 (Estruturas de dados para conjuntos disjuntos) e 23 (Árvores geradoras mínimas).

KLEINBERG, Jon; TARDOS, Éva. **Algorithm Design**. Boston: Pearson/Addison-Wesley, 2006. Cap. 4 (Greedy Algorithms), seções 4.5 a 4.7.

KRUSKAL, Joseph B. On the shortest spanning subtree of a graph and the traveling salesman problem. **Proceedings of the American Mathematical Society**, v. 7, n. 1, p. 48-50, 1956.

TARJAN, Robert E. Efficiency of a good but not linear set union algorithm. **Journal of the ACM**, v. 22, n. 2, p. 215-225, 1975.